

## **WEEK-7**

### **EXPERIMENT-7**

#### **1.Title of the Experiment :**

The Student Course Management System is a basic server-side web application developed using Node.js and Express.js. It manages student information stored in a JSON file and provides HTTP GET routes to retrieve all students, retrieve a student by ID, search students by course, display system information, and perform a DNS lookup.

#### **2. Aim :**

To develop a Student Course Management System using Node.js and Express.js that manages student information stored in a JSON file and provides different HTTP GET routes for retrieving student details, searching students by course, obtaining system information, and performing DNS lookup.

#### **3. Procedure:**

1. Install Node.js on the system.
2. Create a project folder named week-7.
3. Open the project folder in VS Code.
4. Initialize the Node.js project using `npm init -y`.
5. Install Express.js using `npm install express`.
6. Create `app.js`, `students.json`, and the required project configuration files.
7. Store student information in `students.json`.
8. Import the `express`, `fs`, `os`, and `dns` modules.
9. Create an Express application and configure it to run on port 3000.
10. Implement the required GET routes.
11. Start the server using `node app.js`.
12. Open the required URLs in a browser and verify the output.

#### **4. CODE:**

##### **4.1 app.js**

```
const express = require("express"); const
fs = require("fs");
const os = require("os"); const
dns = require("dns");

const app = express(); const
PORT = 3000;

const FILE = "./students.json";
```

```

// GET all students app.get("/students", (req, res) => {
fs.readFile(FILE, "utf8", (err, data) => {    if (err) {
return res.status(500).send("Error reading student data");
    }

    const students = JSON.parse(data);
    res.json(students);
  });
});

// GET student by ID app.get("/students/:id", (req, res) => {
fs.readFile(FILE, "utf8", (err, data) => {    if (err) {
return res.status(500).send("Error reading student data");
    }

    const students = JSON.parse(data);
    const id = parseInt(req.params.id);

    const student = students.find(s => s.id === id);

    if (student) {      res.json(student);
    } else {          res.status(404).send("Student not
found");
    }
  });
});

// GET students by course app.get("/search", (req, res) => {
fs.readFile(FILE, "utf8", (err, data) => {    if (err) {
return res.status(500).send("Error reading student data");
    }

    const students = JSON.parse(data);
    const course = req.query.course;

    const result = students.filter(      student =>
student.course.toLowerCase() === course.toLowerCase()
    );

    res.json(result);
  });
});

// GET system information
app.get("/system", (req, res) => {
res.json({    platform:
os.platform(),    architecture:
os.arch(),    hostname:
os.hostname(),    cpus:
os.cpus().length,    memory:
os.totalmem()
  });
});

// GET DNS information app.get("/dns", (req, res) => {
dns.lookup("google.com", (err, address, family) => {
  if (err) {
    return res.status(500).send("DNS lookup failed");
  }
}
}

```

```

    }
    res.json({
hostname: "google.com",
address: address,      family:
family
    });
});
});

// Start server app.listen(PORT, () => { console.log(`Server
running at http://localhost:${PORT}`);
});

```

## 4.2 students.json

```

[
  {
    "id": 1,
    "name": "Bhuvan",
    "course": "Node.js"
  },
  {
    "id": 2,
    "name": "Mogal",
    "course": "JavaScript"
  },
  {
    "id": 3,
    "name": "JP",
    "course": "Node.js"
  },
  {
    "id": 4,
    "name": "kiran",
    "course": "Express.js"
  },
  {
    "id": 5,
    "name": "Ravi",
    "course": "HTML"
  }
]

```

## 4.3 package.json

```

{
  "name": "week-7",
  "version": "1.0.0",
  "description": "",
  "main": "app.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs",
  "dependencies": {
    "express": "^5.2.1"
  }
}

```

```
}  
}
```

## 5. Technologies Used

| Technology       | Purpose                             |
|------------------|-------------------------------------|
| Node.js          | Server-side JavaScript runtime      |
| Express.js       | Web server and routing framework    |
| JavaScript       | Application programming language    |
| NPM              | Package and dependency management   |
| JSON             | Storage of student information      |
| File System (fs) | Reading student data from JSON file |
| OS (os)          | Retrieving system information       |
| DNS (dns)        | Performing DNS lookup               |
| VS Code          | Development environment             |
| Web Browser      | Testing HTTP routes                 |

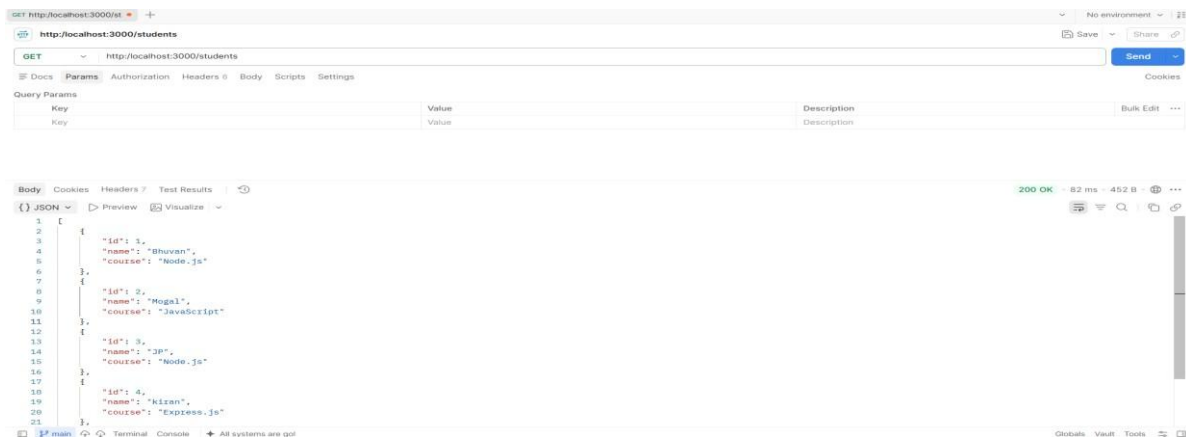
## 6. Folder Structure

```
week-7/ |  
├── app.js  
├── students.json  
├── package.json  
├── package-lock.json  
└── node_modules/
```

## 8. Output

### 8.1 Get All Students

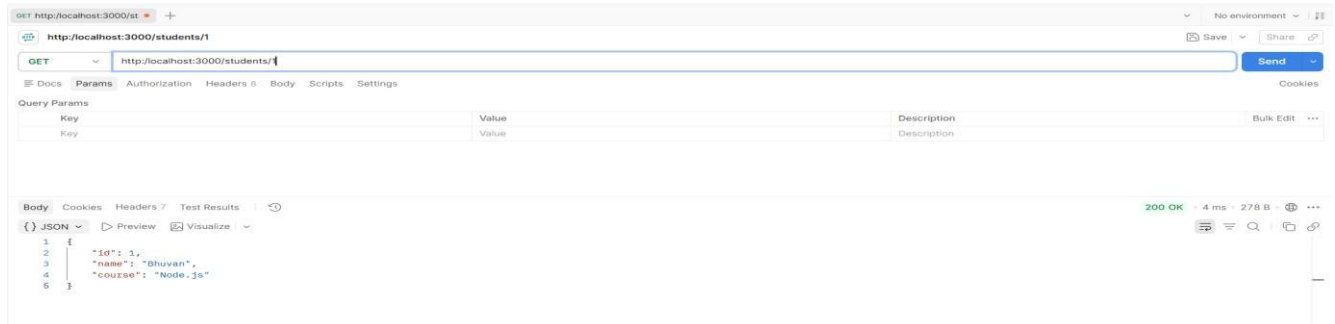
URL: <http://localhost:3000/students>



## 8.2 Get Student by ID

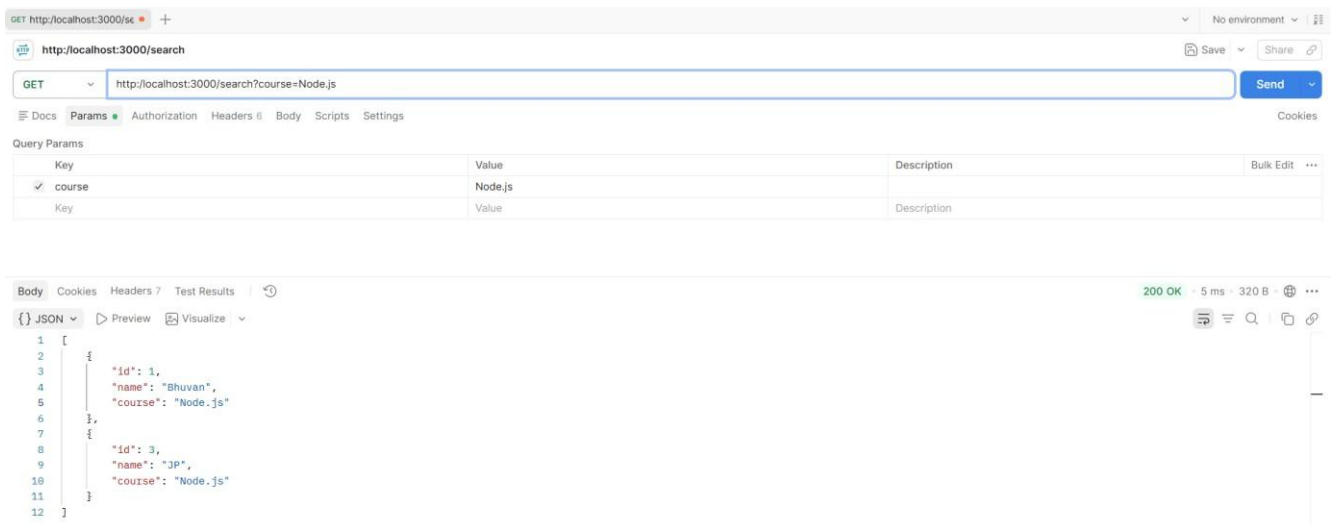
URL: <http://localhost:3000/students/1>

For an invalid ID, the response is: Student not found



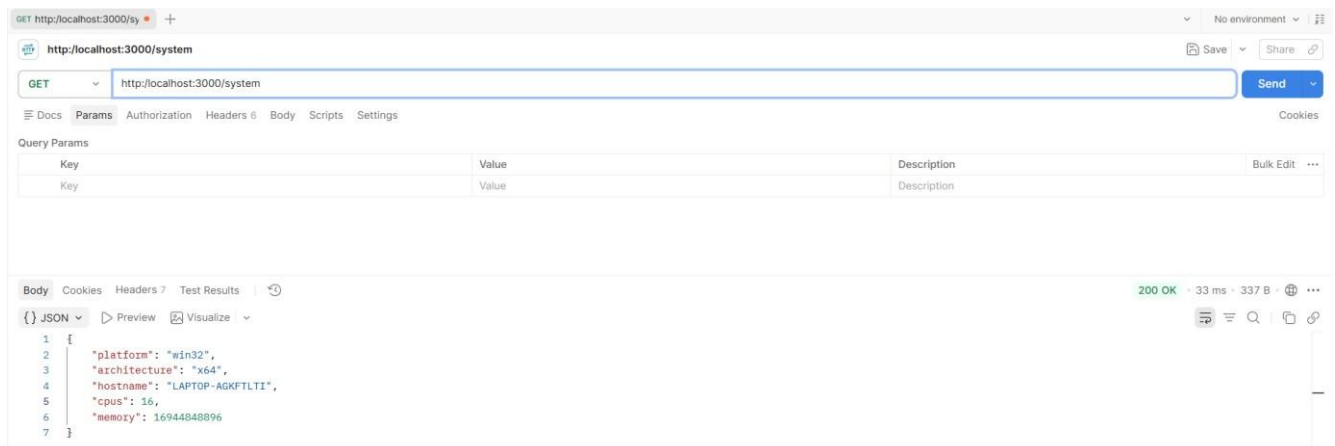
## 8.3 Search Students by Course

URL: <http://localhost:3000/search?course=Node.js>



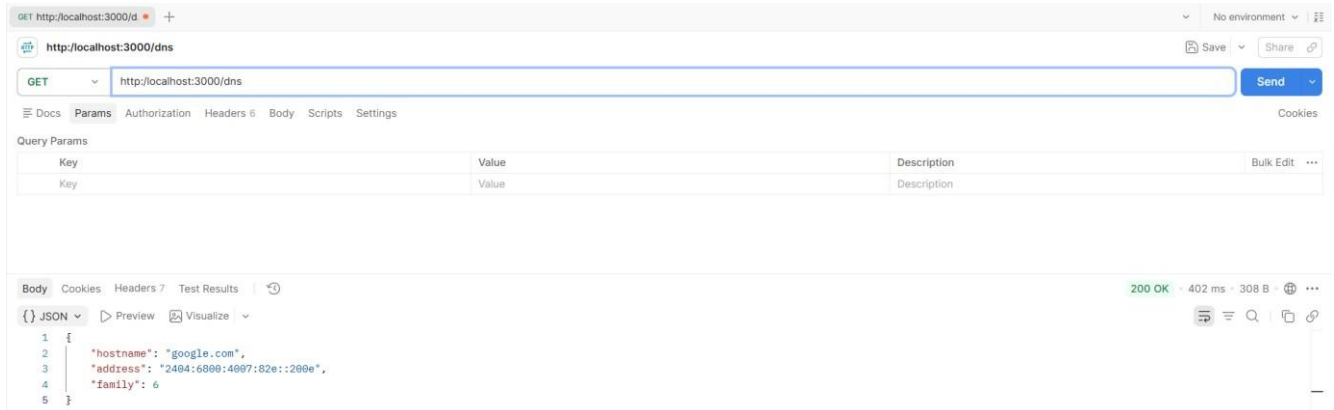
## 8.4 System Information

URL: <http://localhost:3000/system>



## 8.5 DNS Information

URL: <http://localhost:3000/dns>



## 9. Result

Thus, the Student Course Management System was successfully developed using Node.js and Express.js. The application demonstrates Express routing, HTTP GET requests, URL parameters, query parameters, JSON file handling, and Node.js built-in modules such as fs, os, and dns. The required student and system information can be successfully retrieved through the implemented REST endpoints.